

Informe Final del Proyecto: Juego Sweet Crush

Estudiante: Antonio Carlos Merlano Ricardo

Programa: Ingeniería Telecomunicaciones, Universidad de Antioquia

Asignatura: Informática II

1. Introducción y Objetivos

Este documento describe el desarrollo y la implementación del juego "Sweet Crush" en C++, Además complementa el informe preliminar previamente entregado. El proyecto consiste en administrar un tablero dinámico a través de la consola, donde el usuario selecciona las opciones para eliminar fichas, provocando caídas por gravedad y reacciones en cadena (cascadas) cuando se juntan tres o más números iguales.

El desarrollo se enfoca en aplicar conceptos de programación y arquitectura de, cumpliendo con las siguientes restricciones técnicas:

- **Interfaz:** El juego se ejecuta íntegramente en la consola.
- **Manejo de Memoria:** Toda la estructura del tablero debe almacenarse en un único bloque continuo de memoria dinámica utilizando los operadores `new` y `delete[]`.
- **Tipos de Datos:** Está prohibido asignar un entero o un byte completo por cada ficha. Cada casilla debe ocupar exactamente 3 bits en la RAM.

2. Organización del Código

Para evitar errores de compilación cruzada y facilitar la lectura del código, el sistema se diseñó separando las funciones en cinco módulos independientes (.cpp y .h). Cada módulo tiene una responsabilidad única y no interfiere con el trabajo de los demás:

- **main.cpp:** Es el archivo de arranque. Su única tarea es invocar el menú inicial para comenzar la ejecución. Mantener este archivo mas sencillo obtenemos una lógica que asegura el programa inicie sin asignar variables complejas.
- **interfaz.cpp:** Controla todo el flujo de entrada y salida. Agrupa las instrucciones `cin` y `cout` utilizadas para imprimir el tablero y leer las coordenadas que ingresa el usuario. Separar esto evita que las reglas del juego se mezclen con el formato de texto.

- **juego.cpp:** Es el archivo donde residen las reglas matemáticas de la partida. Aquí se ejecutan las funciones para revisar combinaciones, calcular la caída de las fichas, registrar los puntajes y generar los números aleatorios. No maneja punteros directamente.
- **tablero.cpp:** Es el único módulo del programa que interactúa con el sistema operativo para pedir o liberar memoria RAM. Calcula los índices del arreglo y contiene las rutinas para agregar o eliminar filas y columnas durante la partida.
- **bits.cpp:** Funciona en el nivel más bajo del programa. Ejecuta los operadores a nivel de bits (<<, >>, &, |) necesarios para empaquetar los números enteros en fracciones de byte y guardarlos en el bloque de memoria.

3. Administración de Memoria Dinámica

El parámetro más estricto del proyecto fue garantizar la contigüidad absoluta de la memoria. Al programar una cuadrícula, lo normal es crear una matriz dinámica con punteros dobles (`int**`). Sin embargo, crear un arreglo diferente por cada fila fragmentaba la información, obligando al procesador a buscar los datos en sectores mas separados de la RAM, lo cual es ineficiente.

Para solucionar esto, se declaró un único puntero de una dimensión que funciona como un bloque lineal: `unsigned char* memoria = nullptr`; De esta forma, cuando se pide el espacio total con `new unsigned char[bytes]`, el sistema entrega un espacio continuo ininterrumpido.

3.1. Simulación de Coordenadas (Aritmética de Punteros)

Al usar un arreglo unidimensional (1D), la navegación requiere traducir las coordenadas bidimensionales a una posición lineal. Para esto, implementé la función `indice_de`, que calcula la posición exacta aplicando la siguiente fórmula matemática: $\text{Índice Lineal} = (\text{Fila} * \text{Total_Columnas}) + \text{Columna}$. Con este cálculo, el programa imprime una cuadrícula en la pantalla mientras opera sobre una línea recta en la memoria, mejorando el rendimiento de lectura.

3.2. Cambio de Dimensiones y la Regla del 65%

El tablero del juego permite agregar o quitar filas y columnas en cualquier momento. Esto exige modificar el tamaño de la memoria sin perder las fichas que ya están en el juego. La función `reconstruir` ejecuta este proceso en los siguientes pasos:

1. Calcula la nueva cantidad de bytes necesarios para el nuevo tamaño.

2. Reserva un puntero temporal dinámico (`aux = new unsigned char[nuevos_bytes]`).
3. Recorre las fichas del tablero viejo y las copia en el nuevo bloque recalculando sus posiciones.
4. Borra el bloque original usando `delete[]` memoria para evitar fugas de memoria.
5. Reasigna la dirección del puntero principal haciendo `memoria = aux`.

Cuando el usuario elimina partes del tablero, se aplica una regla matemática para no desperdiciar procesamiento: la memoria solo se reduce y se libera si el número de bytes nuevos es menor al 65% de la capacidad que ya está reservada ($\text{bytes_nuevos} * 100 < \text{bytes_actuales} * 65$). Si el tamaño nuevo supera el 65%, el sistema no realiza el cambio y recicla el bloque grande, reordena los datos y pone a cero el espacio sobrante.

4. Manipulación a Nivel de Bits

Los números de las fichas válidas en el juego van del 0 al 5, y se necesitan dos estados adicionales: 110 para los espacios vacíos y 111 para marcar las fichas que se van a eliminar. Esto da un total de 8 estados posibles, lo que significa que cada casilla se puede representar ocupando exactamente 3 bits.

4.1. Cálculo del Tamaño del Arreglo

Para determinar el tamaño real del arreglo `unsigned char`, la función `bytes_necesarios` multiplica el total de posiciones del tablero por 3 y divide el resultado entre 8. Para evitar perder los bits sobrantes cuando la división no es exacta, se le suma 7 a la multiplicación antes de dividir. Esto hace que se fuerce un redondeo hacia arriba, asegurando que se reserve un byte extra de seguridad. Todos los bits válidos se agrupan a la izquierda del bloque.

4.2. Función de Escritura (`escribir_ficha`)

Guardar 3 bits dentro de espacios de 8 bits genera un problema físico: muchas fichas no caben enteras dentro de un solo byte. Para ubicar una ficha, se calcula en qué byte arranca con $(\text{indice} * 3) / 8$ y en qué bit específico empieza con $(\text{indice} * 3) \% 8$.

Si la ficha debe empezar a guardarse en la posición 6 o 7 de un byte, el valor se parte por la mitad. Una parte se guarda al final del byte actual, y el resto cae al principio del byte que sigue. Esto se solucionó utilizando operaciones booleanas en la función `escribir_ficha`:

1. **Primer fragmento:** Se usa una máscara invertida y una compuerta AND (&) para poner en ceros el final del byte actual sin borrar las fichas de los lados. Luego, con un corrimiento a la izquierda (<<) y una compuerta OR (|), se insertan los primeros bits de la ficha.
2. **Segundo fragmento:** Al valor original de la ficha se le aplica un corrimiento a la derecha (>>) para aislar los bits que no cupieron. Estos bits sobrantes se insertan al comienzo de la siguiente dirección (memoria + 1). Estas rutinas también se aplicaron de forma inversa en la función leer_ficha para extraer los datos correctamente.

5. Algoritmos del Juego

El archivo juego.cpp procesa el comportamiento físico de la cuadrícula utilizando ciclos condicionales.

5.1. Caída y Gravedad (aplicar_gravedad)

El programa debe hacer que las fichas bajen cuando se eliminan casillas. Si el recorrido se hace de arriba hacia abajo, las fichas superiores sobrescriben a las de abajo y la información se daña. La solución fue invertir el análisis: el código recorre cada columna desde la base hacia arriba (fila--). Cuando encuentra un valor vacío (el código 110 o 6 en decimal), busca la primera ficha sólida por encima y la mueve a esa posición. Los espacios que quedan arriba se llenan nuevamente con el estado vacío.

5.2. Combinaciones Múltiples (Intersecciones)

Un error lógico durante el desarrollo ocurría al buscar líneas de tres fichas iguales. Si el código borraba la línea apenas la encontraba, rompía las combinaciones que tenían cruces en forma de "L" o "T". Al borrar la fila horizontal, el vértice desaparecía antes de poder evaluar la columna vertical.

Para arreglar esto, el proceso se dividió en dos funciones separadas:

1. **Marcar (marcar_combinaciones):** Las funciones buscar_horizontal y buscar_vertical escanean todo el tablero. Cuando encuentran una combinación, no la borran, sino que cambian el valor de esas fichas por el número 7 (estado especial).
2. **Eliminar (eliminar_marcadas):** Una vez revisado todo el tablero, esta función barre el arreglo y convierte todos los 7 en espacios vacíos (6) al mismo tiempo. Esto ocurre dentro de la función resolver_cascadas, que funciona con un ciclo while. El ciclo baja las fichas, rellena los huecos y vuelve a buscar

combinaciones. Solo se detiene cuando el tablero queda estable, permitiendo sumar puntos adicionales por las reacciones en cadena.

5.3. Generador de Números Matemático

Investigando la fórmula matemática estándar que usan las computadoras para generar números aleatorios. Esto debido a el uso de rand, como restricción por ser heredado de C.

Se implementó la fórmula directamente en el código: $\text{semilla} = \text{semilla} * 1103515245u + 12345u$; $\text{ficha} = (\text{semilla} \gg 16) \% 6$;

Esta multiplicación altera la semilla, y al sacarle el residuo (% 6), entrega siempre un número entre el 0 y el 5 para rellenar las fichas del tablero

6. Validaciones de Interfaz

Uno de los problemas más comunes de la consola C++ ocurre con el canal de entrada cin. Si el menú pide un número de coordenada ($\text{cin} \gg \text{opcion}$) y el usuario escribe una letra, el sistema entra en un estado de error (failbit). Esto hace que el teclado deje de responder y el programa se quede en un ciclo infinito ejecutando la misma instrucción sin parar.

Para arreglar el juego y hacerlo estable, incluí el siguiente bloque en el archivo interfaz.cpp:

C++

```
if (!(cin >> opcion)) {  
    if (cin.eof()) { exit(1); }  
    cin.clear();  
    cin.ignore(10000, '\n');  
}
```

Si el usuario ingresa un dato incorrecto, la función `cin.clear()` reinicia el estado del canal de entrada y borra la advertencia de error. Inmediatamente después, `cin.ignore(10000, '\n')` elimina todos los caracteres que quedaron atorados en la memoria del teclado hasta que encuentra un salto de línea. Así, el programa simplemente vuelve a pedir la coordenada sin cerrarse.

7. Estructura de Seguimiento y Etapas de Desarrollo

Desarrollar este sistema manipulando punteros y bits directamente es complejo y propenso a errores, por lo que el trabajo se estructuró de manera progresiva mediante un archivo de seguimiento de actualizaciones. El desarrollo se documentó en 20 pasos desde un inicio para tener un control de cuantas actualizaciones más o menos se tendrían pensadas y estructuradas y viables, agrupados en cinco fases fundamentales:

Fase 1: Configuración Base y Memoria Dinámica (Actualizaciones 01 a 05) Se inició configurando el archivo del proyecto para consola en C++17. Se crearon los módulos base (main.cpp, bits.cpp, tablero.cpp, juego.cpp, interfaz.cpp) asegurando que cada uno tuviera una responsabilidad distinta. En esta etapa se programó la función `bytes_necesarios` para calcular la cantidad exacta de memoria que ocupan las fichas en fracciones de 3 bits, junto con la función `limpiar_bytes` que recorre el puntero y lo inicializa en cero para evitar valores basura en el arranque.

Fase 2: Lectura y Escritura de Bits (Actualizaciones 06 a 09) El esfuerzo se centró exclusivamente en probar el módulo `bits.cpp`. Se programó primero el escenario ideal, donde los 3 bits de la ficha se leen y escriben dentro de un mismo byte usando la máscara `00000111`. Una vez estable, se abordó el escenario complejo: calcular la fracción que cae en el primer byte y la que cae en el segundo cuando la ficha está partida en la memoria. No se avanzó a la lógica del juego hasta garantizar que estas operaciones no afectaban a los bits vecinos.

Fase 3: Búsqueda y Marcado de Combinaciones (Actualizaciones 15 y 16) Con el arreglo de memoria funcionando, se desarrollaron las rutinas de las reglas del juego. Se programaron de forma independiente las funciones `buscar_horizontal()` y `buscar_vertical()`. Para evitar el problema de las intersecciones, se introdujo la función integradora `marcar_combinaciones()`, seguida de `eliminar_marcadas()`, la cual se encarga de reemplazar todas las fichas marcadas por el valor `ficha_vacia` en un solo recorrido del tablero.

Fase 4: Gravedad, Cascadas y Puntuación (Actualizaciones 17 y 18) Se incorporó el motor de colapso de las piezas. Se diseñó la función `aplicar_gravedad()` con un recorrido desde abajo hacia arriba para evitar sobrescrituras. Luego, se consolidó el ciclo `while` principal en la rutina `resolver_cascadas()`. Esta función mantiene el bucle activo mientras existan combinaciones detectadas, acumulando los contadores de las cascadas y calculando el puntaje final del usuario basándose en la longitud de las reacciones en cadena.

Fase 5: Redimensionamiento y Pulido Final (Actualizaciones 19 y 20) En la fase de cierre, se abordaron las funciones de modificación de la matriz (`reconstruir()`). Se habilitó la capacidad de agregar y eliminar columnas o filas enteras, programando la creación de bloques temporales aux y aplicando la regla del 65% para decidir si se libera y reasigna el puntero memoria o si se reutiliza. Para finalizar, se programó la función `mostrar_estadisticas()` que despliega en pantalla la memoria en uso, movimientos, puntos y cascadas tras cada jugada, completando así el desarrollo del proyecto.

8. Conclusiones

Al terminar el proyecto se puede decir que con dos arreglos dinámicos sí es posible satisfacer el requisito de contigüidad, aunque utilizar un solo arreglo dinámico se mostró como una mejor opción. Al contrario de las matrices normales, los datos del tablero se guardaban en un único espacio de memoria y se trabajaba con varias dimensiones mediante cálculos matemáticos. Esto ayudó a mantener toda la información más organizada en lugar de separarla en diferentes espacios.

Por otro lado, trabajar con solo tres bits por ficha permitió reducir bastante el tamaño de los datos. Uno de los aspectos mas complicados requirió fue manejar la información que podía pasar de un byte a otro. Sin embargo, mediante el uso de máscaras y desplazamientos de bits fue posible organizar estos datos sin perder información.

Por último, el proyecto también sirvió para controlar los datos recibidos desde el teclado y eliminar correctamente las filas y columnas que podían cruzarse. Y logrando aplicar en la práctica los conceptos de memoria dinámica.